

Hola Mundo — Banneton

Tutorial: Next.js + Prisma + PostgreSQL (Supabase) + Docker

Proyecto: Banneton

Universidad Nacional de Colombia — Sede Bogotá
Facultad de Ingeniería — Ingeniería de Sistemas y Computación
Curso: Ingeniería de Software

1. Tecnologías utilizadas

Componente	Tecnología	Versión
Lenguaje	TypeScript	5.x
Framework	Next.js (App Router)	16.x
ORM	Prisma	7.x
Base de datos	PostgreSQL (Supabase)	16
Contenedores	Docker + Docker Compose	—
Estilos	Tailwind CSS	4.x

2. Requisitos previos

Antes de comenzar, verificar que estén instalados:

```
node --version    # Node.js 18+
npm --version     # npm 9+
docker --version  # Docker 24+
```

3. Crear el proyecto Next.js

```
npx create-next-app@latest Banneton \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --no-src-dir \
  --import-alias "@/*"

cd Banneton
```

Este comando genera la estructura base con:

- App Router (app/)
- TypeScript configurado
- Tailwind CSS listo
- ESLint integrado

4. Instalar Prisma y dependencias

```
npm install prisma @prisma/client @prisma/adapter-pg pg @types/pg
npx prisma init --datasource-provider postgresql
```

`prisma init` genera dos archivos clave:

- `prisma/schema.prisma` — define el esquema de la base de datos
- `prisma.config.ts` — configuración del datasource (Prisma 7+)

5. Configurar variables de entorno

Crear el archivo `.env` en la raíz del proyecto:

```
DATABASE_URL="postgresql://USUARIO:PASSWORD@HOST:5432/postgres"
```

Nota: Para Supabase, la URL se obtiene en:

Dashboard → Settings → Database → Connection string → Session pooler

Agregar `.env` al `.gitignore` para no exponer credenciales:

```
echo ".env" >> .gitignore
```

6. Configurar el datasource (Prisma 7+)

En Prisma 7, la URL de conexión se declara en `prisma.config.ts` :

```
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: { path: "prisma/migrations" },
  datasource: {
    url: process.env["DATABASE_URL"],
  },
});
```

Y en `prisma/schema.prisma` , el datasource **no** incluye la URL directamente:

```
generator client {
  provider      = "prisma-client-js"
```

```

    previewFeatures = ["driverAdapters"]
  }

datasource db {
  provider = "postgresql"
}

// Entidad de ejemplo: Rol (control de acceso de usuarios)
model Rol {
  id_rol      Int      @id @default(autoincrement())
  nombre_rol  String   @unique @db.VarChar(30)

  @@map("rol")
}

```

7. Aplicar el esquema a la base de datos

```
npx prisma db push
```

Salida esperada:

 Your database is now in sync with your Prisma schema. Done in 3.21s

Generar el cliente Prisma:

```
npx prisma generate
```

8. Crear el singleton de Prisma

Archivo `lib/prisma.ts` — evita múltiples conexiones en desarrollo:

```

import { PrismaClient } from "@prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";

const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL! });

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

export const prisma =
  globalForPrisma.prisma ?? new PrismaClient({ adapter });

if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;

```

9. Crear la API REST

Archivo `app/api/products/route.ts` :

```
import { NextResponse } from "next/server";
import { prisma } from "@lib/prisma";

// GET /api/products - Lista todos los roles
export async function GET() {
  try {
    const roles = await prisma.rol.findMany({
      orderBy: { id_rol: "asc" },
    });
    return NextResponse.json({ ok: true, data: roles });
  } catch (error) {
    console.error(error);
    return NextResponse.json(
      { ok: false, error: "Error al obtener datos" },
      { status: 500 }
    );
  }
}

// POST /api/products - Crea un nuevo rol
export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { nombre_rol } = body;

    if (!nombre_rol) {
      return NextResponse.json(
        { ok: false, error: "nombre_rol es requerido" },
        { status: 400 }
      );
    }

    const rol = await prisma.rol.create({
      data: { nombre_rol },
    });

    return NextResponse.json({ ok: true, data: rol }, { status: 201 });
  } catch (error) {
    console.error(error);
    return NextResponse.json(
      { ok: false, error: "Error al crear registro" },
      { status: 500 }
    );
  }
}
```

10. Levantar la base de datos con Docker

Archivo `docker-compose.yml` :

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: banneton_db
    restart: unless-stopped
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: PASSWORD
      POSTGRES_DB: banneton
    ports:
      - "5432:5432"
    volumes:
      - banneton_pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  banneton_pgdata:
    name: banneton_pgdata
```

Levantar el contenedor:

```
docker compose up -d
```

Verificar que está corriendo:

```
docker ps -a
```

Salida esperada:

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS
01106ce1a2d9	postgres:16-alpine	"docker-entrypoint.s..."	Up 10 seconds	
0.0.0.0:5432->5432/tcp		banneton_db		

11. Ejecutar el proyecto

```
npm run dev
```

Abrir en el navegador: <http://localhost:3000>

12. Prueba desde Postman

GET — Listar roles

Campo	Valor
Método	GET
URL	http://localhost:3000/api/products

Respuesta esperada:

```
{
  "ok": true,
  "data": []
}
```

POST — Crear un rol

Campo	Valor
Método	POST
URL	http://localhost:3000/api/products
Headers	Content-Type: application/json

Body (raw JSON):

```
{
  "nombre_rol": "admin"
}
```

Respuesta esperada:

```
{
  "ok": true,
  "data": {
    "id_rol": 1,
    "nombre_rol": "admin"
  }
}
```

13. Estructura final del proyecto

```
Banneton/
├─ app/
│   └─ api/
│       └─ products/
│           └─ route.ts      # API REST: GET y POST sobre Rol
└─ layout.tsx
```

```
|   └─ page.tsx           # Página de inicio
└─ lib/
|   └─ prisma.ts          # Singleton Prisma
└─ prisma/
|   └─ schema.prisma       # Modelo de datos (Rol, Usuario, etc.)
└─ Documentacion/
|   └─ Proyecto/
|       └─ Hola_mundo.pdf  # Este tutorial
|       └─ Diagramas/
|           └─ Arquitectura.png # Diagrama de arquitectura
└─ Asignaciones/
|   └─ docker-compose.png  # Evidencia docker ps -a
└─ docker-compose.yml     # BD en Docker
└─ setup.sh               # Script Linux/Mac
└─ setup.bat              # Script Windows
└─ prisma.config.ts       # Config Prisma 7
└─ .env.example           # Plantilla de variables
```